

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM**

ĐỒ ÁN 1
**Xây dựng website hỗ trợ đọc
và ghi nhớ từ vựng tiếng Anh bằng AI**

GV HƯỚNG DẪN:
Trần Thị Hồng Yên

SV THỰC HIỆN:
23521224 Trương Hoàng Phúc

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

We would like to express our sincere gratitude to our supervisor, **Lê Thanh Trọng**, for his guidance and support throughout this project.

We would also like to thank the University of Information Technology, VNU-HCM, for providing the necessary resources and environment for us to complete this course.

Finally, we are grateful for the collaboration and hard work of all team members in making this project a reality.

Table of Contents

1. INTRODUCTION	1
1.1. Necessity of the Topic	1
1.2. System Objectives	2
1.3. Research Scope and Limitations	3
1.4. User Scope and Audience	4
1.5. Methodology	5
1.6. Report Structure	6
2. THEORETICAL BASIS AND TECHNOLOGY	8
2.1. Theoretical Basis	8
2.1.1. Model-View-Controller (MVC) Pattern	8
2.1.2. RESTful API	8
2.1.3. Single Page Application (SPA)	9
2.2. Overview of Related Technologies	9
2.2.1. Frontend	9
2.2.2. Backend	9
2.2.3. Database	9
2.2.4. DevOps & Infrastructure	10
2.3. Rationale for Technology Selection	10
2.3.1. Why PERN Stack (Postgres, Express, React, Node)?	10
2.3.2. Why TypeScript?	10
2.3.3. Why Containerization?	10
Bibliography	12

List of Figures

TÓM TẮT ĐỒ ÁN

“The Everything Shop” is a functional e-commerce web application developed as a semester project for the Object-Oriented Analysis and Design course.

The primary goal of this project is to apply theoretical knowledge of software engineering to build a complete Model-View-Controller (MVC) system. The application allows users to browse products, manage a shopping cart, and place orders. It allows administrators to manage products and view orders.

The system is built using the PERN stack (PostgreSQL, Express, React, Node.js) with TypeScript, ensuring type safety and code maintainability. Docker is used to containerize the application for consistent development environments.

This report documents the analysis, design, and implementation of the system, highlighting the practical application of design patterns and web development best practices.

1. INTRODUCTION

1.1. Necessity of the Topic

The development of an “Online Shop Management Software” was assigned as the core project for the Object-Oriented Analysis and Design course. This topic serves as a fundamental exercise in software engineering, requiring students to translate theoretical concepts into a tangible, functioning system.

While the market is flooded with commercial e-commerce solutions, building one from the ground up offers a unique educational vantage point. It compels us to grapple with essential challenges such as:

- **Data Integrity and Modeling:** Designing a relational database that can robustly handle interconnected entities like Users, Products, Orders, and Reviews without data anomalies.
- **Full-Stack Architecture:** Bridging the gap between a dynamic frontend (React) and a stateless RESTful API (Express), ensuring seamless data flow and user experience.
- **State Management:** Implementing persistent user sessions and shopping cart functionality in a stateless HTTP environment.
- **Security Fundamentals:** Practical application of authentication (JWT) and authorization (RBAC) to protect user data and administrative functions.

Therefore, the necessity of this project lies not in creating a novel product for the market, but in mastering the fundamental patterns and technologies that underpin modern web application development.

1.2. System Objectives

The primary objective is to successfully deliver a web-based “Everything Shop” application that meets the course requirements. This involves dual goals: functioning software and educational mastery.

General Objectives:

- **Application Development:** Build a robust, responsive web application that simulates a typical B2C e-commerce workflow.
- **Architectural Design:** Apply the Model-View-Controller (MVC) pattern to separate concerns, ensuring code maintainability and scalability.
- **Infrastructure:** Demonstrate competence in modern deployment technologies by containerizing the application with Docker.

Specific Objectives:

- **User Account Management:**
 - Guest Registration and Secure Login (JWT).
 - Profile Management (Update details).
 - Role-based Access Control (Customer vs Seller vs Admin).
- **Product Discovery:**
 - Browse Products by Category hierarchy.
 - Search functionality for finding items.
 - View detailed product specifications.
- **Shopping and Checkout:**
 - Persistent shopping cart management.
 - Order creation and simulation of payment processing.

- Coupon code application logic.
- **Management:**
 - Admin dashboard for user and system oversight.
 - Seller tools for managing inventory and order statuses., Delete) operations for Products.
 - Ability to view customer orders and update their status (e.g., Pending -> Shipped).

1.3. Research Scope and Limitations

To ensure the project is achievable within the semester's constraints, we have clearly defined the boundaries of the system.

Research Scope (What is included):

- **Business Model:** The system operates on a Business-to-Consumer (B2C) model. A single entity (the “Shop”) sells products directly to multiple end-users (“Customers”). Multi-vendor marketplaces (C2C) are out of scope.
- **Core Modules:**
 - **Authentication Module:** Registration, Login, Logout, Password Hashing.
 - **Product Module:** Categories, Product Listings, Inventory counts.
 - **Order Module:** Cart management, Order creation, Order History.
 - **Admin Module:** Administrative interface for content management.
- **Technology Stack:** strictly adhered to the PERN stack (Postgres, Express, React, Node) with Docker for environment orchestration.

Limitations (What is excluded):

- **Payment Processing:** The system simulates the checkout process up to the point of “Placing Order”. Integration with real-world payment gateways (Stripe, PayPal, VNPay) is simulated or omitted to avoid financial complexity.
- **Advanced Search:** Product search is implemented with basic string matching, lacking features like fuzzy search or relevance ranking.
- **Shipping and Logistics:** Shipping logic is simplified. Real-time shipping rate calculation and tracking integration are not included.
- **Scalability Testing:** While designed with standard practices, the system is not load-tested for high-concurrency production scenarios.

1.4. User Scope and Audience

The system caters to a specific set of users, each with distinct interaction patterns.

User Scope Analysis:

1. Anonymous Visitor (Guest):

- **Goal:** Product Discovery.
- **Permissions:** Read-only access to home page, product listing, and product details. Restricted from adding items to cart or checking out.

2. Registered Customer:

- **Goal:** Commerce Transaction.
- **Permissions:** Full access to browsing features. Authority to manage personal cart, place orders, and view order history. Authenticated via JWT.

3. Administrator:

- **Goal:** User and System Management.

- **Permissions:** Manage user accounts, access system logs, and generate business reports.

4. Seller:

- **Goal:** Catalog and Promotion Management.
- **Permissions:** Manage products, categories, and coupons.

Target Audience: While simulating a commercial application, the primary consumers of this software and report are:

- **Course Lecturers:** To assess the team's ability to implement a full-stack solution.
- **Peer Students:** To serve as a reference implementation for future cohorts studying Web Application Development.

1.5. Methodology

The project followed a modified Waterfall methodology, adapted for a small team with a fixed deadline. The development lifecycle was divided into distinct phases:

1. Analysis Phase:

- **Requirement Gathering:** Listing core features (Login, Product, Cart).
- **Use Case Modeling:** Identifying actors (Admin, User) and their interactions.

2. Design Phase:

- **Database Schema:** Designing the Entity-Relationship Diagram (ERD) for PostgreSQL, defining tables for User, Product, Order, OrderItem.
- **API Contract:** Defining RESTful endpoints (e.g., GET /api/products, POST /api/cart).
- **Wireframing:** Creating basic UI layouts for the React frontend.

3. Implementation Phase:

- **Backend First:** Built the Express server, Prisma models, and core logic. Tested via Postman.
- **Frontend Integration:** Built React components to consume the API.
- **Containerization:** Added Dockerfiles to ensure the app runs identically on all developer machines.

4. Testing Phase:

- **Unit Testing:** Basic tests for utility functions.
- **Manual Acceptance Testing:** Verifying the end-to-end user flow (Register -> Shop -> Checkout).

Tools & Environment:

- **IDE:** Visual Studio Code.
- **VCS:** Git (Branching strategy: Gitflow Workflow).
- **Project Management:** Basic Kanban board for tracking progress.

1.6. Report Structure

This report documents the entire lifecycle of “The Everything Shop” project. It is organized into 8 chapters:

- **Chapter 1: Introduction** – Outlines the project’s necessity, objectives, scope, and the methodology used for development.
- **Chapter 2: Theoretical Basis and Technology** – Provides a deep dive into the technologies chosen (React, Express, Prisma, Docker) and the rationale behind them.
- **Chapter 3: System Requirements Analysis** – Details the functional specifications, actor interactions, and use case diagrams.

- **Chapter 4: System Design** – Presents the technical blueprints: Application Architecture, Database Design (ERD), and API Design.
- **Chapter 5: Implementation and Deployment** – Describes the actual coding structure, project organization, and how the system is deployed using Docker.
- **Chapter 6: Testing and Evaluation** – Covers the testing strategy, test cases executed, and an evaluation of the system against initial objectives.
- **Chapter 7: Conclusion and Future Development** – Summarizes the project's success, discusses lessons learned, and proposes future enhancements.
- **Chapter 8: References** – Lists all academic and technical resources cited.

2. THEORETICAL BASIS AND TECHNOLOGY

2.1. Theoretical Basis

This chapter outlines the theoretical frameworks and architectural patterns that underpin the design of the “Everything Shop”.

2.1.1. Model-View-Controller (MVC) Pattern

The application is structured using the **MVC architecture**, a design pattern that decouples the user interface (View), data (Model), and application logic (Controller).

- **Model:** Represents the data and business rules. In our system, this is managed by the **Prisma ORM** and the PostgreSQL database, wrapped by a **Service Layer** to handle business logic.
- **View:** The presentation layer that interacts with the user. Our **React** frontend serves as the view, consuming data from the API and rendering the UI.
- **Controller:** The interface between the Model and View. Our **Express** controllers receive HTTP requests, validate input, invoke the Service layer, and return the appropriate JSON response.

2.1.2. RESTful API

Communication between the frontend and backend relies on **REST (Representational State Transfer)** principles.

- **Statelessness:** Each request from the client to the server contains all the information needed to understand and process the request.
- **Resource-Based:** Data is exposed as resources (e.g., /products, /orders) manipulated using standard HTTP methods (GET, POST, PUT, DELETE).

2.1.3. Single Page Application (SPA)

The frontend is built as an SPA, meaning the application loads a single HTML page and dynamically updates content as the user interacts with the app. This provides a smoother user experience similar to a desktop application, avoiding full page reloads.

2.2. Overview of Related Technologies

The following technologies were selected to implement the system:

2.2.1. Frontend

- **React.js:** A JavaScript library for building user interfaces based on components. It allows for the creation of reusable UI elements and efficient state management.
- **Redux:** A predictable state container for JavaScript apps, used here to manage global states like user authentication status and shopping cart contents.

2.2.2. Backend

- **Node.js:** A JavaScript runtime built on Chrome's V8 engine, allowing us to execute JavaScript on the server side.
- **Express.js:** A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
- **Prisma:** A next-generation Object-Relational Mapper (ORM) for Node.js and TypeScript. It is used to define the data schema and interact with the database in a type-safe manner.

2.2.3. Database

- **PostgreSQL:** A powerful, open-source object-relational database system known for its reliability, feature robustness, and performance.

2.2.4. DevOps & Infrastructure

- **Docker:** A platform for developing, shipping, and running applications in containers. It ensures consistency across different development environments.
- **MinIO:** A high-performance, S3-compatible object storage server used for storing product images.
- **MailHog:** A developer tool for testing email sending capability without requiring a real SMTP server.

2.3. Rationale for Technology Selection

The selection of technologies was driven by the need for a modern, scalable, and developer-friendly stack.

2.3.1. Why PERN Stack (Postgres, Express, React, Node)?

- **Unified Language:** Using JavaScript/TypeScript on both the frontend and backend reduces context switching and allows for code reuse (e.g., shared types).
- **Ecosystem:** The Node.js ecosystem (npm) is the largest in the world, providing libraries for almost any functionality required (e.g., authentication, validation).
- **Performance:** Node.js's non-blocking I/O model is well-suited for I/O-heavy applications like e-commerce APIs.

2.3.2. Why TypeScript?

We chose TypeScript over plain JavaScript to enforce static typing. This helps catch errors at compile-time rather than runtime, significantly improving code quality and maintainability, especially in the backend logic.

2.3.3. Why Containerization?

Using Docker and Docker Compose solves the “it works on my machine” problem. By defining the environment (Node version, Database version) in code, every team

member can spin up the entire stack with a single command, ensuring a consistent development experience.

Bibliography